

MCDI500

PROGRAMACIÓN PARA LA CIENCIA DE DATOS

SUMATIVA 1 — FASES 1 Y 2

Análisis reproducible de fallas en maquinaria industrial con AI4I 2020

Nombre del estudiante: Alexander Sepúlveda Ormazábal

Grupo: 9

Docente responsable: Omar Salinas Silva

Fecha: 28 de septiembre de 2026

Repositorio: github.com/alexander-seor/proyecto-grupo9-mcdi500

Índice

1 Introducción y contextualización	3
2 Definición de la problemática y objetivos	3
2.1 Problemática	3
2.2 Pregunta de investigación.....	3
2.3 Objetivo general.....	3
2.4 Objetivos específicos	3
2.5 Alcance, supuestos y limitaciones	4
3 Aplicación de herramientas científicas.....	4
4 Reproducibilidad técnica	5
5 Control de versiones	5
6 Documentación del proceso.....	6
7 Repositorio GitHub	7
8 Codificación funcional	7
9 Desarrollo del notebook F1 Definición	8
10 Preprocesamiento y transformación del dataset.....	9
10.1 Procedencia e integridad	9
10.2 Diagnóstico inicial	9
10.3 Limpieza y selección.....	10
10.4 Codificación y escalamiento	10
10.5 Resultado del pipeline.....	11
11 Validación técnica y verificación del código	11
12 Desarrollo del notebook F2 Preprocesamiento.....	11
13 Vinculación del mapa conceptual con el avance	12
14 Conclusiones.....	14
Referencias	15
Anexo A Matriz de verificación de la rúbrica	16

1 Introducción y contextualización

Las fallas inesperadas en maquinaria industrial pueden interrumpir la operación, reducir la productividad y aumentar las necesidades de mantenimiento. En este contexto, el mantenimiento predictivo busca utilizar mediciones del funcionamiento de los equipos para reconocer condiciones asociadas con una falla antes de que esta produzca una detención. La literatura reciente señala que el monitoreo continuo de componentes y el uso de métodos de aprendizaje automático forman una base relevante para el mantenimiento predictivo en manufactura (Phan et al., 2022).

El proyecto utiliza AI4I 2020 Predictive Maintenance Dataset, un conjunto sintético que representa situaciones de mantenimiento predictivo encontradas en la industria. La fuente oficial informa 10.000 observaciones, seis características principales y ausencia de valores faltantes (Matzka, 2020). Cada registro reúne el tipo de producto, temperaturas, velocidad de rotación, torque, desgaste de herramienta y etiquetas de falla.

El proyecto se organizó en cuatro fases relacionadas, cada una con un propósito definido dentro del flujo de trabajo:

1. Fase 1, definición y orientación: delimitó la problemática, formuló los objetivos, configuró el entorno reproducible y documentó las variables.
2. Fase 2, preparación de los datos: convirtió la planificación en un pipeline de obtención, exploración, limpieza, transformación, validación y persistencia.
3. Fase 3, desarrollo algorítmico: abordará el núcleo del proyecto mediante programación estructurada, recursiva y orientada a objetos.
4. Fase 4, análisis y comunicación: evaluará los resultados mediante análisis, visualizaciones y una comunicación técnica coherente con la problemática.

El alcance de este informe corresponde a las dos primeras fases. Se describen los datos preparados para etapas posteriores, pero todavía no se entrena ni se evalúa un modelo predictivo. Esta separación sigue la guía de la evaluación, que reserva la limpieza y la transformación para la Fase 2 (Universidad Andrés Bello, 2026a).

La programación es necesaria porque el análisis debe ejecutar de forma consistente varias verificaciones sobre 10.000 registros, aplicar las mismas reglas de transformación a entrenamiento y prueba, detectar entradas inválidas y conservar evidencia de cada resultado. Pandas, NumPy y scikit-learn permiten automatizar esas tareas; Jupyter integra código, explicación y salidas; Git y GitHub registran la evolución del trabajo.

2 Definición de la problemática y objetivos

2.1 Problemática

Aunque existen mediciones de las condiciones de operación de una máquina, esas mediciones no indican por sí solas cuáles se relacionan con las fallas. El problema consiste en preparar y estudiar de manera reproducible las variables de AI4I 2020 para identificar condiciones operacionales asociadas con la ocurrencia de fallas y posibles patrones de riesgo. Debido al carácter sintético del conjunto, las asociaciones encontradas deberán interpretarse como evidencia dentro del dataset y no como relaciones causales aplicables directamente a una planta real.

2.2 Pregunta de investigación

¿Qué condiciones operacionales están asociadas con la ocurrencia de fallas en maquinaria industrial y en qué medida estas variables permiten identificar situaciones de riesgo de falla?

2.3 Objetivo general

Analizar la relación entre las condiciones operacionales de la maquinaria industrial y la ocurrencia de fallas mediante el conjunto de datos AI4I 2020, con el propósito de identificar posibles patrones de riesgo y evaluar posteriormente la capacidad de estas variables para anticipar fallas.

2.4 Objetivos específicos

1. Caracterizar las variables operacionales y la variable de falla presentes en AI4I 2020.
2. Evaluar la calidad inicial del conjunto mediante dimensiones, tipos, valores faltantes, duplicados, rangos y distribución de la variable objetivo.

3. Preparar un conjunto consistente y reproducible, justificando las decisiones de limpieza, selección y transformación aplicadas en la Fase 2.
4. Explorar posteriormente la relación entre temperatura, velocidad de rotación, torque, desgaste de herramienta, tipo de producto y ocurrencia de fallas.
5. Evaluar en las fases siguientes si las variables operacionales permiten reconocer situaciones con mayor riesgo de falla.

2.5 Alcance, supuestos y limitaciones

El trabajo incluye la definición del problema, la verificación del entorno, la documentación y validación inicial del dataset, la construcción del pipeline de Fase 2 y la generación de conjuntos procesados. Quedan fuera de esta entrega la interpretación causal, la validación en maquinaria real, el entrenamiento de modelos y el despliegue de un sistema de mantenimiento.

Los resultados se interpretan bajo los siguientes supuestos:

- Las definiciones, unidades y roles publicados por UCI representan correctamente las variables del archivo descargado.
- Machine failure se utiliza como objetivo binario y cada fila representa una observación operacional válida.
- El archivo original se conserva sin cambios y su hash permite comprobar su identidad.
- La división estratificada conserva una proporción comparable de fallas en entrenamiento y prueba.

Las principales limitaciones son:

- AI4I 2020 es sintético y no corresponde a una planta o máquina industrial específica.
- Solo 339 de los 10.000 registros contienen una falla, por lo que la variable objetivo está desbalanceada.
- El dataset no aporta una fecha y hora industrial real ni un historial completo de intervenciones de mantenimiento.
- Los resultados no prueban causalidad ni aseguran su validez fuera del dataset.

3 Aplicación de herramientas científicas

Las herramientas se seleccionaron por la función que cumplen dentro del pipeline. No se utilizaron como demostraciones aisladas: cada biblioteca participa en una etapa que conecta la entrada, el procesamiento y la evidencia producida.

Herramienta	Uso aplicado
NumPy	Semilla, operaciones numéricas y comprobaciones estadísticas.
Pandas	Carga del CSV, perfilado, copias, limpieza, tablas y persistencia.
Matplotlib	Gráficos de distribución y control del escalamiento.
scikit-learn	OneHotEncoder, división estratificada y comparación de escaladores.
pathlib	Rutas relativas independientes de la carpeta personal.
hashlib	Hash SHA-256 para verificar la integridad del archivo original.
json	Exportación de metadatos anidados y decisiones técnicas.

Tabla 1. Herramientas utilizadas y propósito dentro del proyecto

Las operaciones principales se encapsularon en funciones como `buscar_raiz`, `validar_columnas`, `cargar_dataset`, `validar_valores_permitidos`, `detectar_atipicos_iqr`, `limpiar_datos`, `codificar_one_hot`, `validar_dataset` y `aplanar`. Esta organización reduce repeticiones y permite probar cada responsabilidad por separado.

4 Reproducibilidad técnica

La reproducibilidad se incorporó desde el inicio para que el proyecto pudiera ejecutarse nuevamente sin depender de la sesión en que se construyeron los notebooks. Para ello se controlaron el entorno de Python, las versiones de las librerías, la semilla, las rutas de los archivos y la integridad del conjunto original. Estas comprobaciones quedaron visibles dentro de F1 y F2, de modo que cada ejecución deja evidencia del contexto utilizado.

El entorno virtual `.venv` concentra las dependencias del proyecto y evita mezclar librerías instaladas para otros trabajos. `requirements.txt` registra las versiones utilizadas, mientras que `sys.executable` permite comprobar que Jupyter está usando el intérprete de ese entorno. Este procedimiento sigue el criterio de la guía del curso: otro equipo debería poder recrear el entorno y obtener el mismo resultado

Componente	Versión
Python	3.13.0
NumPy	2.5.3
pandas	3.0.5
Matplotlib	3.11.2
scikit-learn	1.9.1

Tabla 2. Componentes principales del entorno verificado

Las rutas se construyeron a partir de la raíz del repositorio. La función `buscar_raiz` localiza `README.md`, `requirements.txt` y la carpeta `F1` antes de formar las rutas hacia los datos y documentos. Así, el código puede ejecutarse desde la raíz o desde la carpeta de un notebook sin incorporar una dirección personal como `C:/Users`. `Path.cwd()` se muestra en pantalla para reconocer de inmediato la carpeta desde la que se inició la ejecución.

La semilla 42 se declaró antes de cualquier operación aleatoria y se reutilizó en la división de entrenamiento y prueba. Su función es conservar la misma selección de registros cuando el pipeline vuelve a ejecutarse. De esta manera, una diferencia posterior puede atribuirse a un cambio en el código o en los datos y no a una nueva partición generada al azar.

```
SEMILLA = 42
np.random.seed(SEMILLA)
RAIZ = buscar_raiz(Path.cwd())
ARCHIVO_CRUDO = RAIZ / 'F1' / 'data' / 'raw' / 'ai4i2020.csv'
```

Fragmento 1. Semilla y construcción de rutas reproducibles

La integridad del archivo original se verificó mediante el hash SHA-256

`dc6630cd9b1f0f853922fad78a1b6436570d3f1ec863f1dd5c4340ac56bc8a8e`. Este valor funciona como una huella del CSV: la modificación de un solo byte produce un hash diferente. La comparación permite confirmar que `F2` procesa el mismo archivo que fue documentado en `F1` y que los resultados no provienen de una copia alterada.

En conjunto, estas medidas permitieron reiniciar el kernel y ejecutar los notebooks en orden continuo sin corregir rutas ni completar variables manualmente. La reproducibilidad queda respaldada por la configuración del entorno, por las comprobaciones impresas y por los archivos de metadatos generados al finalizar cada fase.

5 Control de versiones

Git se utilizó para registrar estados sucesivos del proyecto y GitHub para mantener una copia remota verificable. El procedimiento aplicado en cada avance siguió esta secuencia:

1. Revisar el estado del repositorio y reconocer los archivos modificados o nuevos.
2. Preparar únicamente los archivos relacionados con el avance que se iba a registrar.
3. Crear un commit descriptivo que resumiera el cambio realizado.
4. Ejecutar push y comprobar la sincronización con el repositorio remoto.

Al cerrar la Fase 2, la rama local main y origin/main apuntaban al commit d6cd7b2. Para los cambios posteriores se mantuvo una convención de mensajes con los siguientes prefijos:

- docs: cambios en documentación, informes o archivos explicativos.
- feat: incorporación de una funcionalidad o producto nuevo.
- fix: corrección de un error o de un comportamiento no esperado.

Commit	Descripción
d6cd7b2	Actualiza verificación final de F2.
be64efc	Completa preprocesamiento de F2.
ed3da7b	Completa definición y validación reproducibles de F1.
093e41b	Actualiza README y entregables de Formativa 1.
f75a1eb	Documenta dataset y validación inicial de variables.
64ff7b8	Agrega dataset AI4I 2020.
d5d135e	Registra avances de Fase 1 y dependencias.
8e9c38e	Crea el repositorio.

Tabla 3. Historial principal de commits

El trabajo fue desarrollado por un solo integrante. Por esa razón, el historial contiene una autoría individual y no se simuló contribuciones, ramas o revisiones de compañeros inexistentes. El flujo con ramas y pull requests se comprende como práctica colaborativa y queda disponible para un escenario de equipo o para cambios aislados en fases posteriores.

6 Documentación del proceso

La documentación se construyó junto con el código para conservar el razonamiento de cada etapa. Antes de cada bloque de código, una celda Markdown explica su propósito y su relación con el objetivo del proyecto. Los comentarios se reservaron para decisiones que podían resultar poco evidentes. La evidencia se organizó en los siguientes componentes:

- Fase 1: problemática, objetivos, fuente del dataset, diccionario de variables, evaluación del conjunto, vinculación con el mapa conceptual y metadatos. Estos archivos documentan el origen de AI4I 2020 y el rol analítico de sus catorce variables.
- Fase 2: diagnóstico del dataset, decisiones de limpieza y selección, codificación de Type, escalamiento, división estratificada, resumen de transformaciones, diccionario actualizado y metadatos en CSV y JSON.
- Evidencia de ejecución: versiones HTML de los notebooks con sus explicaciones, código y salidas, complementadas por el README con la estructura del repositorio, la activación del entorno y el orden de ejecución.

La tabla resume las decisiones principales y la evidencia que las respalda. La documentación permite seguir el recorrido desde el dato original hasta los archivos procesados y comprender por qué se tomó cada decisión sin depender únicamente de la lectura del código.

Decisión	Evidencia y efecto
No imputar	Se observaron 0 valores faltantes; imputar habría modificado datos completos.
No eliminar filas	Se observaron 0 duplicados exactos; se conservaron los 10.000 registros.
Excluir identificadores	UDI y Product ID identifican registros y productos, pero no condiciones operacionales.
Excluir modos de falla	TWF, HDF, PWF, OSF y RNF están ligados directamente a la etiqueta y producirían fuga de información.

Decisión	Evidencia y efecto
Conservar atípicos	Se detectaron 418 en velocidad y 69 en torque; podrían representar situaciones operacionales relevantes.
Usar RobustScaler	La mediana y el IQR reducen la influencia de valores extremos sin eliminarlos.
Dividir antes de escalar	El escalador aprende solamente desde entrenamiento y se aplica a prueba sin reajustarse.

Tabla 4. Decisiones técnicas y evidencia utilizada

7 Repositorio GitHub

El repositorio se encuentra en <https://github.com/alexander-seor/proyecto-grupo9-mcdi500>. La raíz contiene README.md, requirements.txt y .gitignore. F1 conserva el archivo original, el notebook de definición, el módulo utilidades y la documentación inicial. F2 contiene el notebook de preprocesamiento, los conjuntos procesados y los documentos generados.

Ruta	Función
F1/data/raw	Conserva ai4i2020.csv sin modificaciones.
F1/notebooks	Contiene F1_Definicion.ipynb y validador.ipynb.
F1/src	Contiene utilidades.py con funciones reutilizables.
F1/docs	Guarda diccionario, evaluación, mapa, metadatos y HTML.
F2	Contiene F2_preprocesamiento.ipynb.
F2/data/processed	Guarda entrenamiento y prueba procesados.
F2/docs	Guarda HTML, diccionario, metadatos y resumen de transformaciones.

Tabla 5. Estructura principal y función de cada ubicación

El README explica la problemática, el dataset, la estructura, la activación del entorno, el orden de ejecución, las decisiones de F1 y F2 y el estado actual. .gitignore excluye .venv, cachés y archivos temporales. requirements.txt registra las versiones efectivamente instaladas.

8 Codificación funcional

Cada función se diseñó con parámetros claros y una responsabilidad principal. Los DataFrame se copian antes de transformarse, por lo que df_crudo permanece disponible como referencia. Las validaciones detienen la ejecución cerca del origen del problema mediante ValueError, KeyError y assert con mensajes comprensibles.

Función	Entrada	Resultado
buscar_raiz	Ruta inicial	Raíz del repositorio.
cargar_dataset	Ruta y columnas	DataFrame verificado.
detectar_atipicos_iqr	Serie numérica	Límites y cantidad de atípicos.
limpiar_datos	Datos crudos	Siete columnas seleccionadas y tipadas.
codificar_one_hot	Datos y nombre de columna	Datos codificados, codificador y nombres.
validar_dataset	Dataset y nombre	Confirmación de estructura e integridad.
aplanar	Diccionario anidado	Metadatos en un solo nivel.

Tabla 6. Funciones principales del flujo

```
X_entrenamiento, X_prueba, y_entrenamiento, y_prueba = train_test_split(
    X, y, test_size=0.20, random_state=SEMILLA, stratify=y
)
escalador_final.fit(X_entrenamiento[COLUMNAS_OPERACIONALES])
escalador_final.transform(X_prueba[COLUMNAS_OPERACIONALES])
```

Fragmento 2. División reproducible y control de fuga de información

La función recursiva aplanar vuelve a llamarse cuando encuentra un diccionario y termina al recibir un valor simple. Su comportamiento se comprobó mediante los siguientes casos:

- Estructura simple, con una clave y un valor terminal.
- Estructura anidada, con un diccionario dentro de otro.
- Estructura profunda, para verificar llamadas recursivas sucesivas.
- Diccionario vacío, tratado como caso límite.
- Lista como valor terminal, conservada sin alterar su contenido.

9 Desarrollo del notebook F1 Definición

F1_Definicion.ipynb establece el alcance conceptual y técnico del proyecto sin limpiar ni transformar los datos, tareas reservadas para F2. El notebook contiene 25 celdas narrativas y 25 celdas de código con ejecución numerada de 1 a 25. Las celdas Markdown explican la finalidad de cada bloque antes de su ejecución, mientras que las salidas conservan evidencia de las comprobaciones realizadas.

El desarrollo del notebook siguió una secuencia verificable:

1. Definición del problema: reunió en la configuración PROYECTO la problemática, las preguntas, el objetivo general, los objetivos específicos, el alcance y las limitaciones.
2. Verificación del entorno: mostró el intérprete activo, la carpeta de trabajo y las versiones de Python, NumPy, pandas, Matplotlib y scikit-learn.
3. Reconocimiento de la estructura: localizó la raíz mediante rutas relativas y comprobó la existencia de las carpetas y los archivos principales del repositorio.
4. Configuración reproducible: revisó .gitignore y requirements.txt, declaró la semilla 42 y evitó rutas personales dependientes de un equipo.
5. Módulo reutilizable: creó o reutilizó F1/src/utilidades.py, lo importó y probó buscar_raiz y validar_columnas en casos normal, límite y de excepción.
6. Evaluación del dataset: cargó AIAI 2020 y comprobó ocho requisitos del curso relacionados con dimensiones, roles analíticos, tipos de variables, nulos, tamaño del archivo y duplicados.
7. Diccionario e integridad: documentó las 14 variables, comparó el diccionario con las columnas reales y verificó 10.000 filas, 14 columnas, 0 valores faltantes y 0 duplicados.
8. Control y validación final: consultó la configuración de Git y ejecutó una función que comprobó ocho carpetas, seis archivos, la definición del proyecto, el dataset, el diccionario, la evaluación y el módulo utilidades.
9. Vinculación y persistencia: relacionó diez elementos del mapa conceptual con evidencias del repositorio y guardó los documentos generados por la fase.

La evaluación obtuvo estado OK en los ocho criterios mínimos del curso. La validación principal confirmó que la estructura, los archivos reproducibles, la definición del proyecto, el dataset, el diccionario, la evaluación y el módulo reutilizable estaban disponibles y eran coherentes. Las pruebas del módulo también mostraron una salida normal, la aceptación de listas vacías como caso límite y una excepción controlada cuando faltó la columna Torque [Nm].

La ejecución dejó los siguientes productos verificables:

- F1/docs/F1_Definicion.html, versión consultable del notebook con narrativa, código y salidas.
- F1/docs/diccionario_variables.csv, con las 14 variables y su rol analítico.
- F1/docs/evaluacion_criterios_dataset.csv, con el resultado de los ocho requisitos del curso.
- F1/docs/vinculacion_mapa_conceptual.csv, con diez elementos y su evidencia asociada.
- F1/docs/metadatos_fase1.json, con entorno, semilla, sistema y datos básicos del conjunto.

Esta organización mantiene correspondencia entre el notebook, el informe y el repositorio. También permite revisar F1 sin ejecutar tareas propias del preprocesamiento y deja una entrada documentada para la Fase 2.

10 Preprocesamiento y transformación del dataset

10.1 Procedencia e integridad

AI4I 2020 fue descargado desde UCI y se mantuvo en F1/data/raw/ai4i2020.csv. La ficha oficial lo describe como sintético, multivariado, con 10.000 instancias y sin valores faltantes (Matzka, 2020). El hash del archivo se calculó antes de procesar y la carga verificó existencia, registros y columnas esperadas.

10.2 Diagnóstico inicial

El diagnóstico encontró 0 valores faltantes, 0 filas duplicadas exactas y 0 identificadores repetidos. Type contiene 6.000 productos L, 2.997 M y 1.003 H. Machine failure contiene 9.661 observaciones sin falla y 339 con falla. Las fallas equivalen al 3,39 %, por lo que una predicción constante de la clase mayoritaria alcanzaría 96,61 % de exactitud sin detectar ninguna falla. Esta cifra muestra por qué las fases posteriores deberán considerar sensibilidad, precisión, F1 y área bajo la curva ROC.

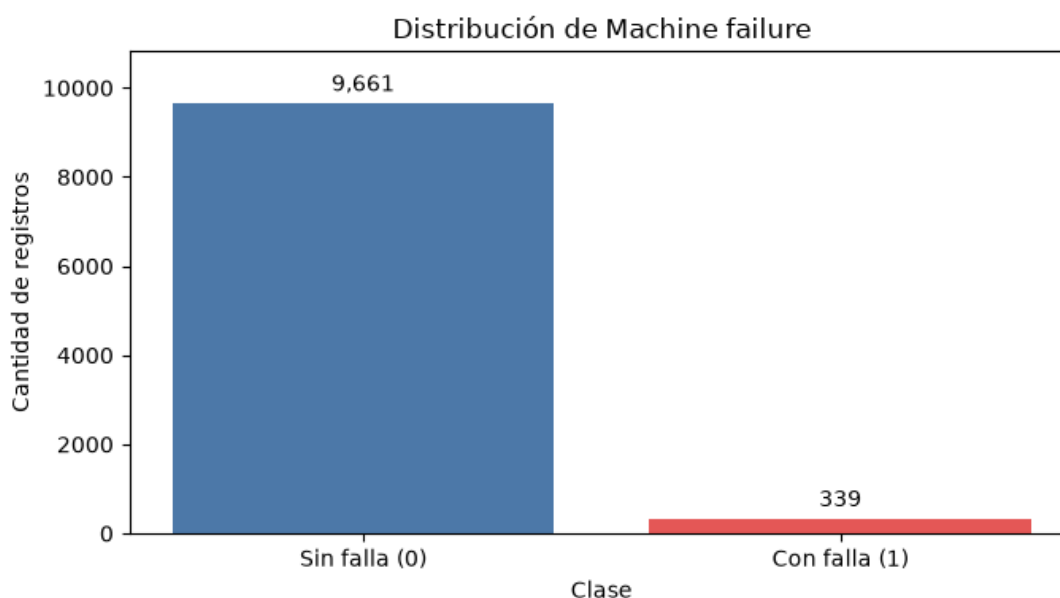


Figura 1. Distribución de la variable Machine failure

Nota. Elaboración propia a partir de AI4I 2020.

El criterio IQR no encontró atípicos en las temperaturas ni en desgaste. Detectó 418 registros en velocidad de rotación, equivalentes al 4,18 %, y 69 en torque, equivalentes al 0,69 %. No se eliminaron porque esos valores pueden representar condiciones relacionadas con una falla.

Variable o grupo	Rol	Tratamiento
UDI y Product ID	Identificadores	Excluir de los predictores.
Type	Nominal	Codificar como Type_H, Type_L y Type_M.

Variable o grupo	Rol	Tratamiento
Temperaturas	Continuas	Escalar con RobustScaler.
Rotational speed	Continua	Escalar y conservar atípicos.
Torque	Continua	Escalar y conservar atípicos.
Tool wear	Discreta	Escalar con RobustScaler.
Machine failure	Objetivo binario	Conservar sin escalar.
TWF, HDF, PWF, OSF y RNF	Modos de falla	Excluir para evitar fuga de información.

Tabla 7. Resumen de variables y tratamiento aplicado

10.3 Limpieza y selección

Como no había nulos ni duplicados, no se aplicó imputación ni eliminación de filas. Tampoco se utilizaron la media, la mediana o la imputación por grupos porque el diagnóstico confirmó 0 valores nulos y no era necesario reemplazar datos completos. Type se normalizó con eliminación de espacios y conversión a mayúsculas, luego se validó contra L, M y H y se convirtió a categoría. Machine failure se convirtió a int8. La tabla analítica pasó de 14 a 7 columnas al excluir identificadores y modos de falla.

10.4 Codificación y escalamiento

Type se transformó con OneHotEncoder en tres columnas binarias. La documentación oficial indica que esta técnica representa cada categoría mediante una columna, lo que permite usar variables categóricas en muchos estimadores sin asignarles un orden artificial (scikit-learn developers, s. f.-a). La comprobación verificó que cada fila tuviera exactamente una de las tres columnas activa.

Se compararon tres alternativas de escalamiento antes de seleccionar la que se aplicaría a las variables operacionales:

- StandardScaler se descartó porque utiliza la media y la desviación estándar, estadísticas sensibles a valores extremos.
- MinMaxScaler se descartó porque depende del mínimo y el máximo, los cuales pueden verse afectados por observaciones extremas.
- RobustScaler se seleccionó porque utiliza la mediana y el rango entre cuantiles, estadísticas menos sensibles a esos valores (scikit-learn developers, s. f.-b).

La división se realizó antes de ajustar el escalador: 8.000 registros se destinaron a entrenamiento y 2.000 a prueba, con random_state=42 y estratificación por Machine failure. RobustScaler se ajustó únicamente con entrenamiento y luego se aplicó a prueba para evitar fuga de información.

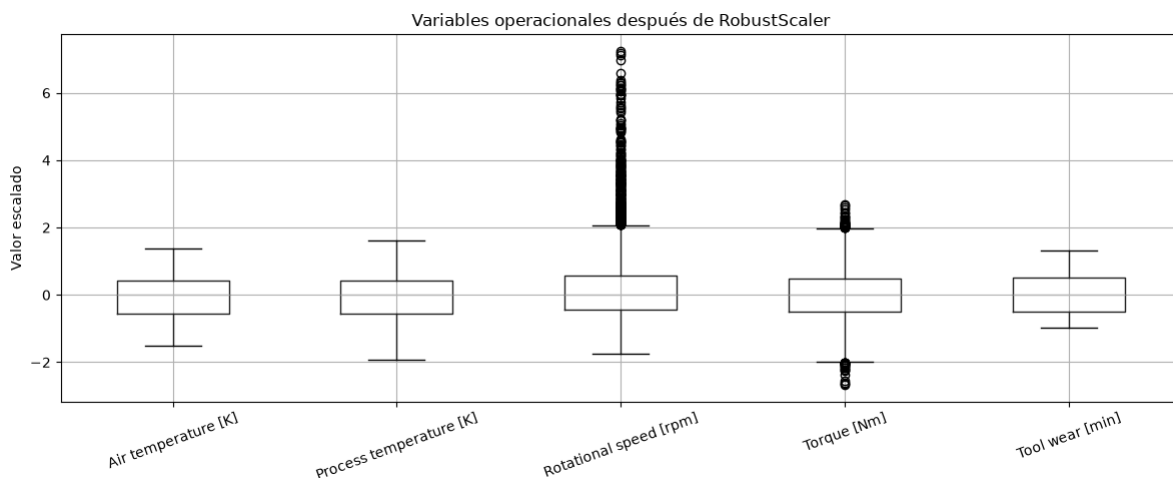


Figura 2. Variables operacionales de entrenamiento después de RobustScaler

Nota. Las medianas quedan cerca de 0 y los IQR cerca de 1; los puntos extremos se conservan.

10.5 Resultado del pipeline

Etapa	Filas	Columnas	Nulos	Resultado
Datos originales	10.000	14	0	Archivo preservado.
Limpieza y selección	10.000	7	0	Variables analíticas.
Codificación One-Hot	10.000	9	0	Type en tres columnas.
Entrenamiento escalado	8.000	9	0	RobustScaler ajustado aquí.
Prueba escalada	2.000	9	0	Transformación sin reajuste.

Tabla 8. Dimensiones y nulos por etapa

11 Validación técnica y verificación del código

La validación final comprobó que entrenamiento y prueba no estuvieran vacíos, no tuvieran nulos, contuvieran solo variables numéricas, conservaran la variable objetivo, tuvieran las columnas esperadas y mantuvieran una codificación Type coherente. También se verificó que ambos conjuntos no compartieran índices y que juntos conservaran los 10.000 registros.

Tipo	Entrada probada	Resultado esperado y observado
Normal	Dataset de entrenamiento procesado	Validación correcta: 8.000 filas por 9 columnas.
Límite	Un solo registro con Type=L	Matriz 1 por 3 y una sola categoría activa.
Excepción	Columna inexistente	KeyError capturado con nombre de la columna.
Excepción	Torque con un valor nulo	ValueError capturado: 1 valor faltante no esperado.
Recursividad	Diccionarios simple, profundo, vacío y con lista	Cinco pruebas superadas.
Persistencia	Relectura de CSV procesados	Dimensiones, columnas y ausencia de nulos confirmadas.

Tabla 9. Pruebas normales, límite y de excepción

Las pruebas no se limitaron a afirmar que el código funcionaba. Cada una produjo una salida visible y las excepciones fueron provocadas deliberadamente para confirmar que el pipeline rechaza una entrada incorrecta.

12 Desarrollo del notebook F2 Preprocesamiento

El notebook F2_preprocesamiento.ipynb convirtió la planificación de F1 en un flujo ejecutable. Cada bloque fue precedido por una explicación de su propósito y de la decisión que se esperaba comprobar. Después de reiniciar el kernel se ejecutaron 23 celdas de código en orden continuo y sin errores. El recorrido se organizó en las siguientes etapas:

1. Obtención e integridad: cargó el CSV conservado en F1/data/raw y comprobó su existencia, sus columnas y su hash.
2. Diagnóstico: confirmó 10.000 registros y 14 variables, sin valores nulos ni duplicados exactos. También registró 339 fallas frente a 9.661 casos sin falla, desbalance que condicionará las métricas de las fases de modelado.
3. Limpieza y selección: conservó las 10.000 filas, excluyó UDI y Product ID por su función identificadora y retiró TWF, HDF, PWF, OSF y RNF de los predictores por estar ligados directamente a la etiqueta de falla. Type se normalizó y se codificó en tres columnas binarias.

4. División y transformación: produjo 8.000 registros de entrenamiento y 2.000 de prueba con `random_state=42`. `RobustScaler` se ajustó únicamente con entrenamiento y después se aplicó a prueba. Los valores atípicos de velocidad de rotación y torque se conservaron por su posible relación con las fallas.
5. Validación: comprobó dimensiones, tipos, columnas, ausencia de nulos, codificación de Type e independencia de índices. Incluyó casos normales, límite y de excepción, entre ellos una columna inexistente y un valor faltante introducido deliberadamente.
6. Persistencia: guardó los conjuntos procesados en `F2/data/processed` y la versión HTML, el diccionario, los metadatos y el resumen de transformaciones en `F2/docs`. Los CSV se leyeron nuevamente para confirmar dimensiones, columnas y valores esperados.

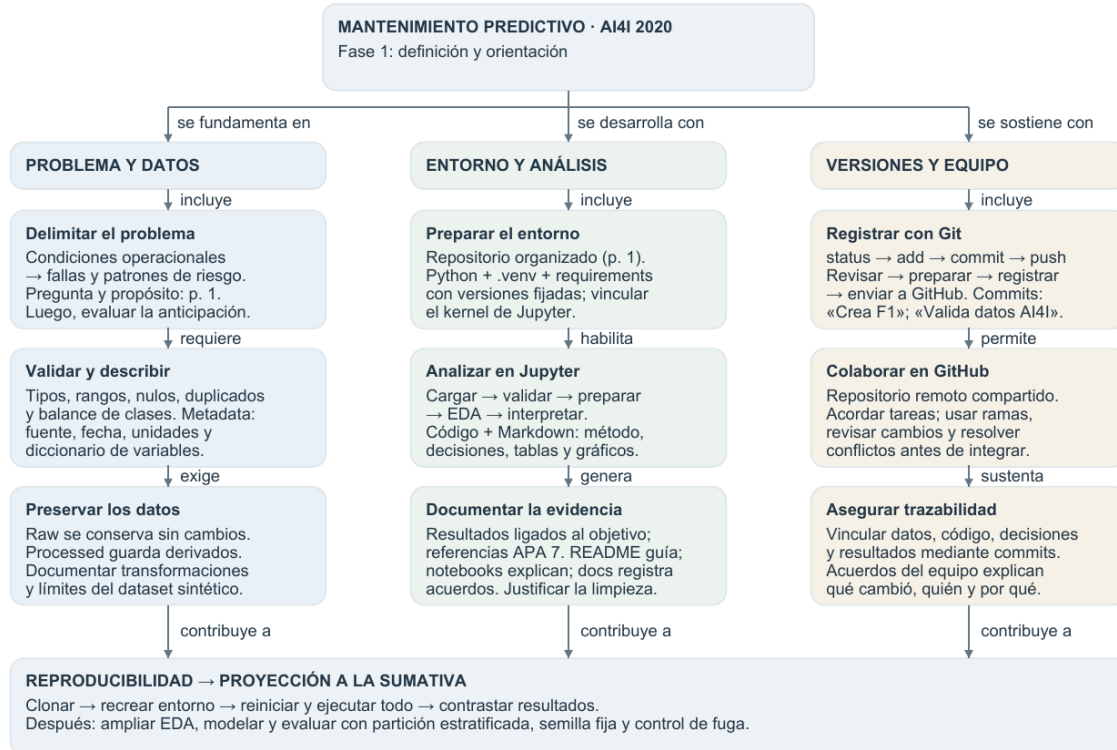
13 Vinculación del mapa conceptual con el avance

El mapa conceptual de la Formativa 1 funcionó como una guía para ordenar el trabajo antes de programar. En las fases 1 y 2, la planificación se transformó en archivos, funciones y resultados revisables dentro del repositorio. La correspondencia se organizó a partir de sus tres ramas principales:

- Problema y datos: se refleja en la problemática, la pregunta y los objetivos de F1; en la ficha de AI4I 2020; en el diccionario; en el diagnóstico de calidad; y en la conservación del archivo raw. Los conjuntos procesados de F2 representan el resultado de preparar los datos sin perder su procedencia.
- Entorno y análisis: se concretó mediante `.venv`, `requirements.txt`, la semilla 42 y las rutas relativas. Jupyter reunió explicación, código y salidas; las funciones separaron las tareas de carga, validación, limpieza, codificación y persistencia; y las tablas y figuras respaldaron las decisiones.
- Versiones y equipo: se vinculó con el historial de Git, la sincronización con GitHub y la documentación de cada avance. Como el proyecto fue desarrollado por un solo integrante, la colaboración se adaptó a un flujo individual sin agregar contribuciones inexistentes.

La figura conserva la visión general planteada al inicio y la tabla relaciona cada elemento con una evidencia concreta. Los componentes destinados a preparar y validar los datos ya se encuentran implementados. El modelado y la evaluación permanecen proyectados para las fases 3 y 4, utilizando los conjuntos preparados y métricas adecuadas para el desbalance de Machine failure.

Mapa del flujo reproducible



Flechas: relaciones entre conceptos · Fuente y glosario: página 1

02

Figura 3. Mapa conceptual del flujo reproducible del Grupo 9

Nota. Elaboración propia presentada en la Formativa 1.

Elemento del mapa	Estado	Evidencia
Delimitar el problema	Implementado	Problemática, pregunta, objetivos, alcance y limitaciones en F1.
Validar y describir	Implementado	Perfil de 10.000 por 14, diccionario y evaluación del dataset.
Preservar los datos	Implementado	Raw sin cambios, hash SHA-256 y derivados en F2.
Preparar el entorno	Implementado	.venv, requirements, kernel verificado, rutas relativas y semilla 42.
Analizar en Jupyter	Implementado	Notebooks F1 y F2 ejecutados y exportados a HTML.
Documentar la evidencia	Implementado	README, docs, metadatos, tablas, gráficos y este informe.
Registrar con Git	Implementado	Ocho commits y sincronización de main con origin/main.
Colaborar en GitHub	Adaptado	Trabajo individual; ramas y pull requests quedan como flujo comprendido.
Modelar y evaluar	Proyectado	Fases 3 y 4; métricas adecuadas para el desbalance.

Tabla 10. Correspondencia entre el mapa y la implementación

14 Conclusiones

Las fases 1 y 2 dejaron una base reproducible para continuar el proyecto de mantenimiento predictivo. La problemática, la pregunta y los objetivos quedaron alineados con las variables disponibles. El entorno, las rutas, las dependencias y la semilla se documentaron, y el historial de Git permite reconstruir el avance.

El diagnóstico mostró que el archivo tiene buena integridad estructural: 10.000 registros, 14 variables, sin nulos ni duplicados exactos. La principal dificultad analítica es el desbalance de Machine failure, con 339 fallas y 9.661 casos sin falla. También se detectaron valores potencialmente atípicos en velocidad y torque, los cuales se conservaron porque podrían contener información relevante.

El pipeline produjo conjuntos de entrenamiento y prueba con nueve columnas numéricas, sin valores faltantes y sin compartir registros. La exclusión de identificadores y modos de falla, la división antes del escalamiento y el ajuste de RobustScaler solo con entrenamiento reducen riesgos de información irrelevante y fuga de datos. Las pruebas normales, límite y de excepción entregan evidencia de que el código responde correctamente ante entradas válidas e inválidas.

La siguiente etapa deberá evaluar asociaciones y modelos con métricas apropiadas para clases desbalanceadas. Los resultados deberán interpretarse dentro de los límites de un dataset sintético; el buen funcionamiento del pipeline no garantiza su generalización a maquinaria industrial real.

Referencias

- Matzka, S. (2020). *AI4I 2020 Predictive Maintenance Dataset* [Conjunto de datos]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5HS5C>
- Phan, T. L. J., Gehrhardt, I., Heik, D., Bahrpeyma, F., & Reichelt, D. (2022). A systematic mapping study on machine learning techniques applied for condition monitoring and predictive maintenance in the manufacturing sector. *Logistics*, 6(2), 35. <https://doi.org/10.3390/logistics6020035>
- Scikit-learn developers. (s. f.-a). *OneHotEncoder*. Scikit-learn documentation. Recuperado el 28 de septiembre de 2026, de <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
- Scikit-learn developers. (s. f.-b). *RobustScaler*. Scikit-learn documentation. Recuperado el 28 de septiembre de 2026, de <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html>
- Universidad Andrés Bello. (2026a). *Guía de desarrollo de la Evaluación Sumativa 1 Fases 1 y 2* [Documento de apoyo docente]. MCDI500 Programación para la Ciencia de Datos.
- Universidad Andrés Bello. (2026b). *Guía de Git y GitHub para Windows* [Documento técnico de apoyo]. MCDI500 Programación para la Ciencia de Datos.

Anexo A Matriz de verificación de la rúbrica

Criterio	Evidencia principal	Estado
Portada e índice	Portada institucional e índice automático.	Cumplido
Introducción	Sección 1 con contexto, relevancia y fases.	Cumplido
Problemática y objetivos	Sección 2 con preguntas, alcance, supuestos y límites.	Cumplido
Herramientas científicas	Sección 3 y funciones de los notebooks.	Cumplido
Reproducibilidad	Entorno, versiones, semilla, rutas y hash.	Cumplido
Control de versiones	Historial de ocho commits y origin/main actualizado.	Cumplido
Documentación	Markdown, comentarios, docs y decisiones justificadas.	Cumplido
Repositorio	README, estructura F1/F2 y dependencias.	Cumplido
Codificación funcional	Funciones, copias y excepciones controladas.	Cumplido
Preprocesamiento	Diagnóstico, limpieza, One-Hot y RobustScaler.	Cumplido
Validación	Casos normales, límite, excepciones y persistencia.	Cumplido
Notebook F1	Ejecutado, documentado y exportado.	Cumplido
Notebook F2	23 celdas ejecutadas, documentado y exportado.	Cumplido
Mapa conceptual	Figura 3 y Tabla 10.	Cumplido
Aspectos formales	Carta, Arial, APA 7, tablas y figuras uniformes.	Cumplido

Tabla A1. Cobertura de los criterios de la Sumativa 1